

LECTURE 19

Deadlocks: Characterization and the Resource-Allocation Graph

Two cars on a bridge, neither willing to back up

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Define deadlock precisely, with a real-world analogy and a concrete system example
- State the four necessary conditions for deadlock, and why all four must hold at once
- Read and draw a Resource-Allocation Graph (RAG): processes, resources, instances, edges
- Distinguish when a cycle in a RAG guarantees deadlock from when it is merely a warning

DEFINITION

The Deadlock Problem

A set of processes is deadlocked when every process in the set is waiting for an event — almost always, the release of a resource — that only another process in that same set can cause.

A Concrete System Example

Process P1 holds resource R1 and is waiting to acquire R2, while process P2 holds R2 and is waiting to acquire R1. P1 cannot proceed until P2 releases R2, and P2 cannot proceed until P1 releases R1 — and neither process will ever release what it holds, because releasing only happens after a process finishes, and finishing is exactly what each process cannot do while blocked.

NOTE

Deadlock is not starvation. A starved process can still eventually be favored by the scheduler (aging). A deadlocked process has no such possibility — it requires outside intervention (Lecture 21) or must be avoided in advance (Lecture 20).

All Four Must Hold Simultaneously

1. Mutual Exclusion

At least one resource is held in a non-shareable mode — only one process can use it at a time

2. Hold-and-Wait

A process holds at least one resource while waiting to acquire additional resources held by others

3. No Preemption

Resources cannot be forcibly taken away; only released voluntarily once a process has finished

4. Circular Wait

A cycle of processes $P_1 \dots P_n$ exists where each waits for a resource held by the next, closing the loop

WARNING

Satisfying three of the four conditions guarantees nothing — deadlock requires the fourth as well. A system with mutual exclusion, hold-and-wait, and no preemption but no circular wait is not deadlocked. This is exactly why the Resource-Allocation Graph is built to detect circular wait directly.

NOTATION

Reading a Resource-Allocation Graph



Process — drawn as a circle

R1



Resource type — a rectangle, with one dot per instance

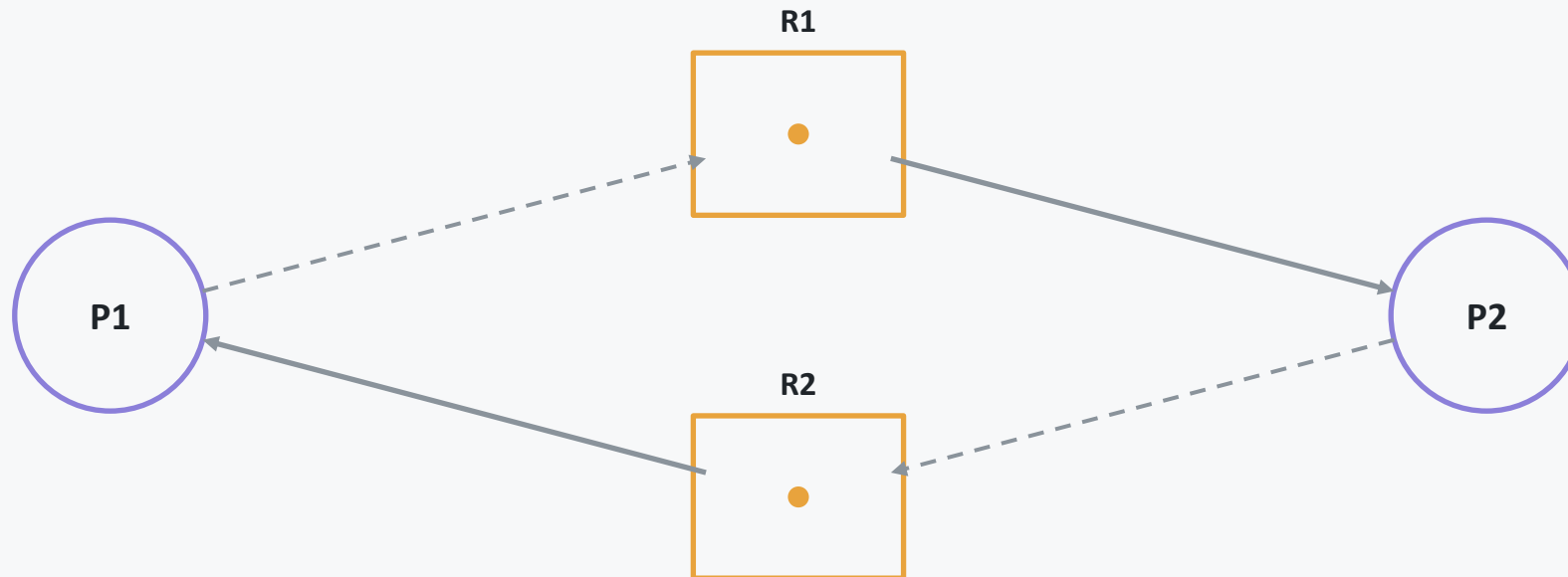


Request edge (dashed): process is waiting to acquire the resource



Assignment edge (solid): one instance of the resource is allocated to the process

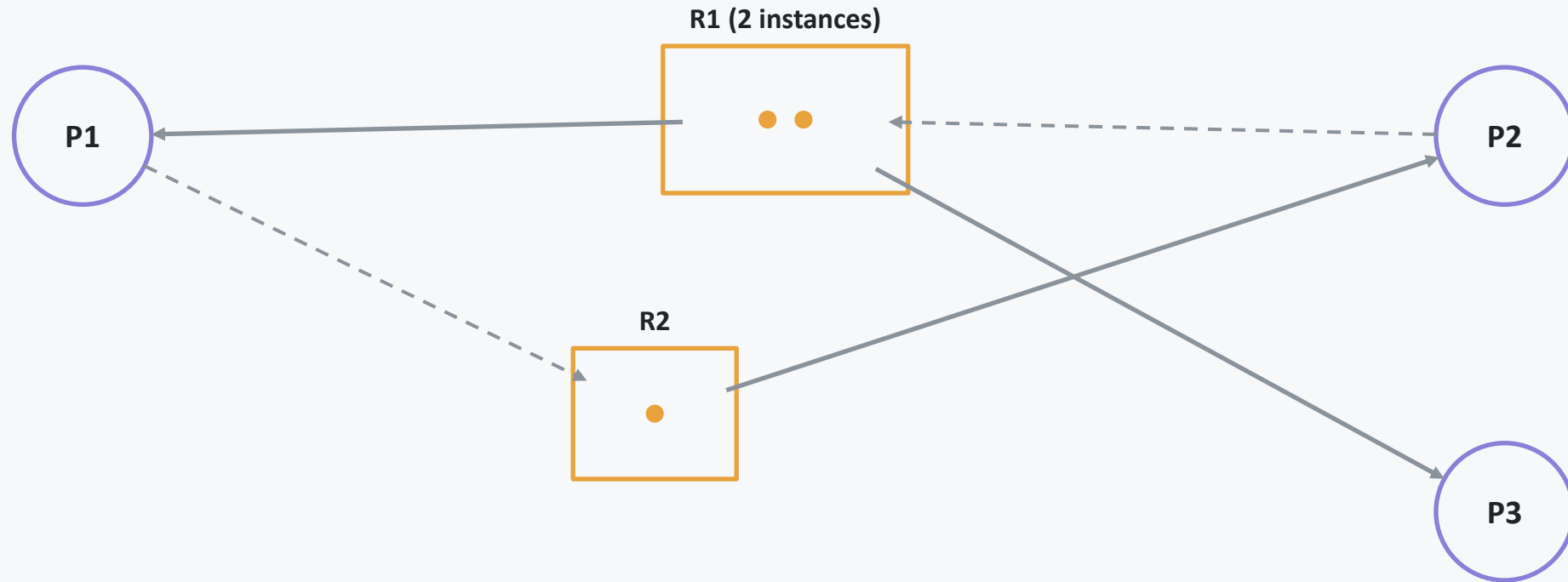
Single-Instance Cycle — Deadlock Guaranteed



Dashed = request edge · Solid = assignment edge

Cycle: $P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow P1$. Every resource type in the cycle has exactly one instance, so no third process can intervene — deadlock is certain.

Multi-Instance Cycle — Not Necessarily Deadlock



Dashed = request edge · Solid = assignment edge

Cycle $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$ exists, but R1's second instance is held by P3, which isn't waiting on anything. P3 can finish and release it — the cycle unwinds with no deadlock.

Interpreting Cycles in a RAG

	Every Resource in Cycle Has 1 Instance	Some Resource Has ≥ 2 Instances
Verdict	Cycle guarantees deadlock	Cycle is necessary but not sufficient
Why	No third process can free the needed instance	An uninvolved process may hold and release another instance
What to check	Nothing further — the cycle alone proves it	Whether an instance outside the cycle can still be released

TIP

The exam-ready version: count the instances of every resource type in the cycle. If all are 1, stop — deadlock. If any is ≥ 2 , check whether a process outside the cycle holds and can release an instance.

KEY TAKEAWAYS

Spotting Deadlock on Sight

1

Deadlock is a set of processes each waiting for an event only another process in that same set can cause — unlike starvation, it cannot resolve itself

2

The four necessary conditions — mutual exclusion, hold-and-wait, no preemption, circular wait — must all hold at once

3

A RAG draws processes as circles, resource types as rectangles with one dot per instance, requests as dashed edges, assignments as solid edges

4

A cycle where every resource type has a single instance guarantees deadlock; a cycle with a multi-instance resource is only a warning sign

Lecture 20 takes the opposite strategy — avoidance — built around the Banker's Algorithm.